

Lernziele, hier erfahren Sie,

- wie Software funktioniert und wie diese mit *maschinennahen* oder *höheren* Programmiersprachen entwickelt werden kann.
- welche wesentlichen Bestandteile Software identifiziert und deren Interaktionen.

Ausgangssituation:

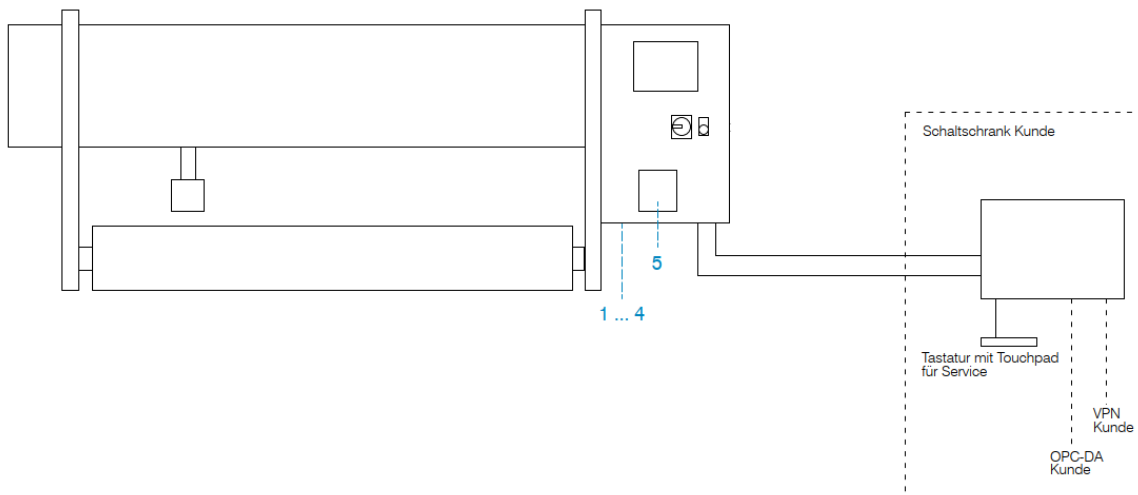
Was hat das mit Softwareentwicklung zu tun?

Warum ist Softwareentwicklung schwierig?

Wie können diese Probleme beherrschbar gemacht werden?

Welche Ziele und Mittel verfolgt somit das Software Engineering?

Stellen Sie sich vor, Sie sind als späterer Fachinformatiker in einem Messtechnik-Unternehmen beschäftigt, das Anlagen zur **Dickenmessung** herstellt. Im Zuge des mechanischen und elektrotechnischen Aufbaus müssen auch die **Schnittstellen** zwischen den einzelnen Teilsystemen (Mechanik, Schaltschrank => Elektrik und PC/Computer) berücksichtigt werden. Ihre **Aufgabe** ist es nun die Anlage PC-gerecht auszustatten, die richtigen **PC-Komponenten** zur Verfügung zu stellen und auszuwählen.



Dabei stellt das **HMI** immer die Bedienerschnittstelle zur Anlage dar. Im Bild oben sehen Sie, wie so eine **Schnittstellenbeschreibung** nun aussehen könnte.

Der PC ist entsprechend mit dem HMI am Schaltschrank der Anlage verbunden. Am PC wiederum werden Maus, Tastatur etc. angeschlossen, um die Anlage über die **Software bedienen** zu können und die **Dicke des Materials** über verschiedene Auswerteverfahren messen zu können.

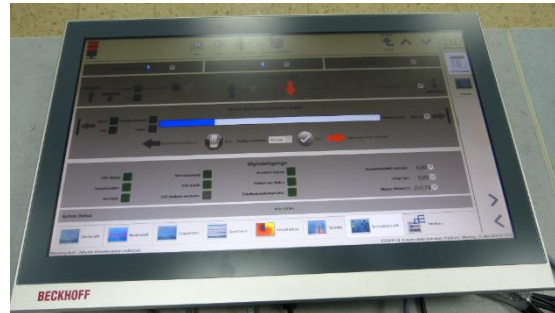
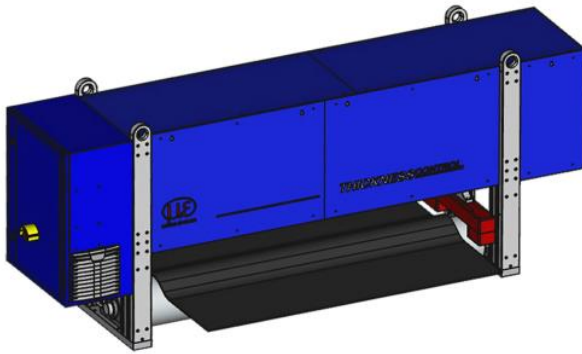


Machen Sie sich Notizen!

Standardsoftware: Bestandteile von Software

Aufbau und Funktionsweise von Software

1	Not-Halt Kunde	4	Erdung
2	Digital Ein/Aus Kunde	5	Druckluftanschluss
3	230V 10A Kunde	---	nicht im Lieferumfang enthalten



Was ist Software? Um welche Art von Software handelt es sich hier?

Was ist Software?

Welche Systematik von Software kommt hier zum Einsatz?

Welche Vor- und Nachteile haben unterschiedliche Softwarearten?



Machen Sie sich Notizen!

Aufbau und Funktionsweise von Software

Welche Hardware umfasst die Ihnen vorliegende Industrieanlage?

Welche Software könnte die vorstehende Industrieanlage beinhalten?

Hardware

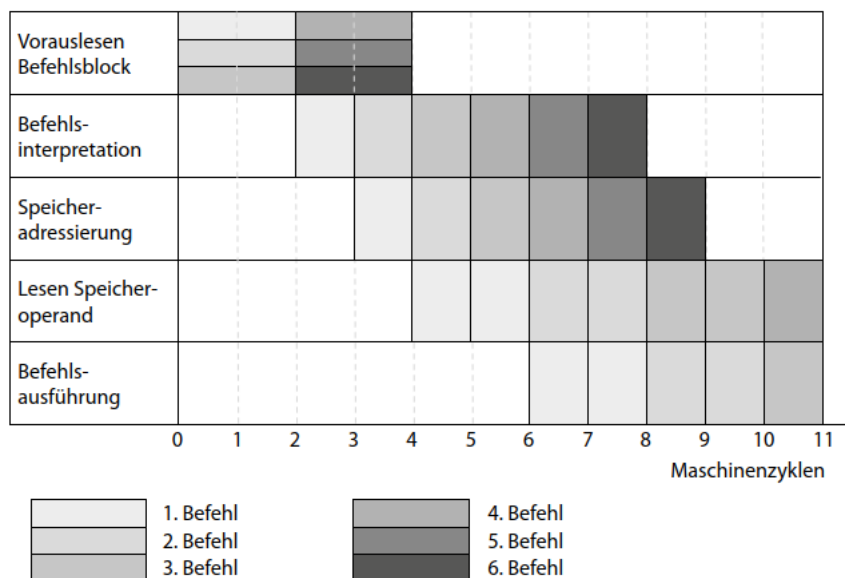
= alle physisch, materiell angreifbaren Komponenten

Software

= alle immateriellen Anteile, welche die Funktionsweise der Industrieanlage steuern

Was ist Software und wie entsteht Software?

Dazu muss man wissen, wie ein **Prozessor** funktioniert.



Wie Sie erkennen können, ist es möglich, dass von einem Prozessor mehrere Maschinenbefehle gleichzeitig verarbeitet werden.

In unserem Beispiel können maximal **drei Befehle** zeitgleich aus dem Zentralspeicher gelesen werden, wobei der erste Befehl nach acht Maschinenzyklen abgeschlossen ist, der zweite nach zehn Zyklen.

Ohne Fließbandverarbeitung hätte die Abarbeitung des zweiten Befehls erst nach dem Abschluss des ersten beginnen können, und er könnte erst zum Zeitpunkt 18 abgeschlossen werden. Nach dem vierten Maschinenzyklus könnten bereits weitere Befehlsverarbeitungen beginnen.

Was passiert hier?

1. Ein auf einem Rechner ausführbares Programm besteht aus einer Reihe von detaillierten Anweisungen an den Rechner in Form von **Maschinenbefehlen**.
2. Damit ein Programm (kann nicht nur *Anweisungen*, sondern auch *Daten* oder *Speicheradressen* beinhalten) ausgeführt werden kann, muss es zuerst in den Arbeitsspeicher geladen werden.
3. Das Maschinenprogramm ist letztendlich eine Folge von **Nullen und Einsen**, die vom Prozessor interpretiert werden.

4. Wenn Menschen Programme als Folge von *binären* Zeichen erstellen würden, wäre dies sehr **aufwendig und fehleranfällig**. Programme müssen von Menschen gelesen und verstanden werden, um diese weiterzuentwickeln, um *Fehler* zu erkennen und *Veränderungen* in den Programmen durchführen zu können.

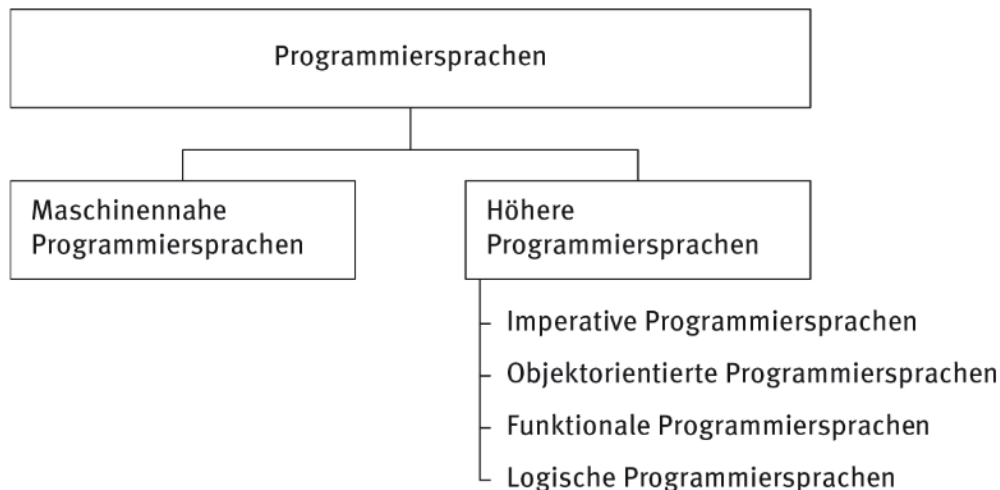
Warum glauben Sie, sollte Software wartbar und anpassbar sein?

Was könnte man bei der vorstehenden Industrieanlage unter einer kundenspezifischen Softwareanpassung verstehen?

Welchen Mehrwert hat das für die Softwareentwicklung?

Programmiersprache

Eine Programmiersprache ist deshalb eine Sprache zur Formulierung von Rechenvorschriften (Algorithmen), die von einem Computer ausgeführt werden können. Mittels Programmiersprachen werden komplexe Aufgabenstellungen durch eine detaillierte Folge von Rechenanweisungen und Datenbeschreibungen in einer automatisierbaren Weise definiert.



Höhere Programmiersprachen sind weitgehend unabhängig von dem zugrundeliegenden **Prozessortyp**. Während ein Quellprogramm in maschinennaher Sprache sich auf Eigenschaften des Prozessors bezieht (beispielsweise Befehlssatz und Register), fehlen diese Elemente in höheren Programmiersprachen völlig.

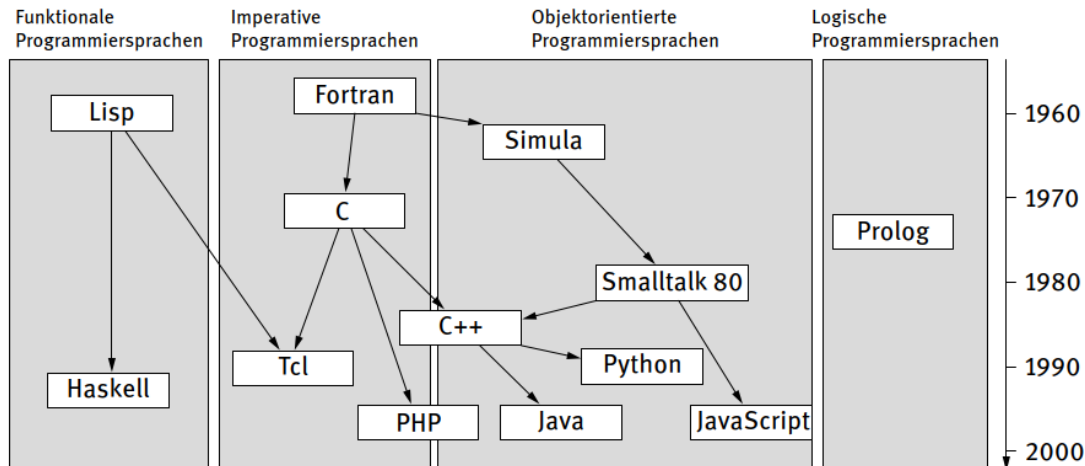
Stattdessen werden dort **Variable und Konstante** in unterschiedlichen Datentypen definiert und für die **Ablaufsteuerung** und Berechnungen werden prozessorunabhängige Befehle verwendet.

Wie könnten sich die o. g. höheren Programmiersprachen nun unterscheiden?

Machen Sie eine Kurz-Recherche zu folgenden Begriffen: imperative Programmiersprachen, objektorientierte Programmiersprachen, funktionale Programmiersprachen, logische Programmiersprachen

Standardsoftware: Bestandteile von Software

Aufbau und Funktionsweise von Software



Wie wird nun ein Quellprogramm aus einer höheren Programmiersprache übersetzt?

- Eine Sprache, deren Befehle sich am Anwendungsbereich orientieren, wird als **höhere Programmiersprache** (engl.: higher level programming language) bezeichnet. Höhere Programmiersprachen abstrahieren vom Befehlssatz des Prozessors und verwenden Elemente der natürlichen Sprache und/oder der Mathematik für das Formulieren von Programmen.
- Ein **Compiler** (auch: Kompilierer, engl.: compiler) ist ein Übersetzungsprogramm, das ein in einer höheren Programmiersprache abgefasstes Quellprogramm in eine Maschinensprache übersetzt (kompiliert). Die Übersetzung erfolgt häufig über den Zwischenschritt der Übersetzung in eine maschinennahe Sprache, wobei dann ein Assembler zur Erzeugung des Maschinencodes herangezogen wird.
- Ein **Binder** (engl.: linker) ist ein Dienstprogramm, das mehrere kompilierte Teilprogramme zu einem gemeinsamen Binärprogramm zusammenfasst und dabei Verweise (wie beispielsweise Funktionsnamen) auf externe Softwarekomponenten durch Sprungadressen ersetzt. Gleichzeitig werden auch Speicheradressen von Variablen ermittelt und im Programm eingesetzt.
- Ein **Lader** (engl.: loader) ist ein Dienstprogramm, das ein Binärprogramm von einem externen Speichermedium liest und in den Arbeitsspeicher kopiert, Kommandozeilenargumente übergibt und Register initialisiert, und schließlich das Programm startet (an die Startadresse des Programms verzweigt).
- Ein **Interpreter** (engl.: interpreter) ist ein Programm, das ein in einer höheren Programmiersprache abgefasstes Quellprogramm bei der Ausführung laufend interpretiert, das heißt, die Anweisungen in dem Programm laufend überprüft, analysiert und entsprechende Operationen vornimmt. Meist wird dabei aus Effizienzgründen beim Laden das Quellprogramm in einen **Zwischencode** (engl.: intermediate code) übersetzt, der interpretiert wird, wodurch wiederholte Überprüfungen von Anweisungen reduziert werden können.

Was könnten in diesem Kontext nun Softwarekomponenten sein?

Lesen Sie sich obige Folie gut durch und erörtern Sie für sich die Frage, was in diesem Kontext Softwarekomponenten sein könnten.

Welche Softwarekomponenten könnten bei unserer Industrieanlage zum Einsatz kommen?

Standardsoftware: Bestandteile von Software

Aufbau und Funktionsweise von Software

Softwarekomponenten

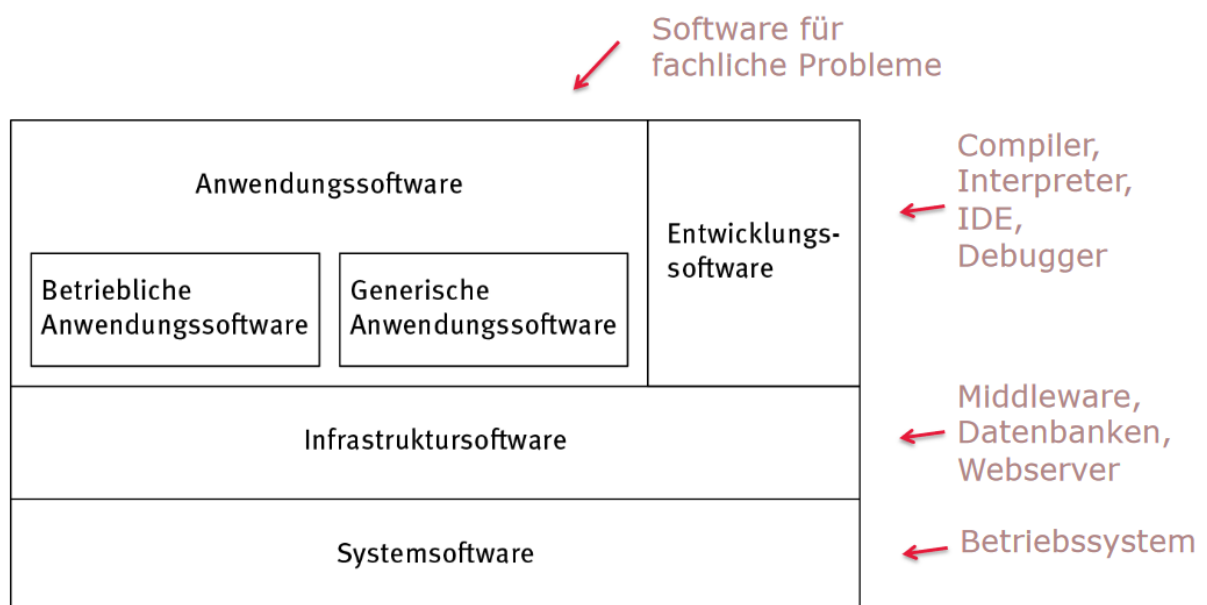
In komplexen Programmsystemen treten immer wiederkehrende Aufgabenstellungen auf, die man aus Gründen der Fehler- und Aufwandsreduktion am besten an einer Stelle möglichst allgemein löst, auch weil man später diese Lösung möglichst oft **wiederverwenden** möchte.

Quellprogramme können sich über Namen auch auf Funktionen beziehen, die nicht in dem Quellprogramm definiert sind.

Auf diese Weise kann ein Programm auf andere **Softwarekomponenten** (beispielsweise *Programmbibliotheken* für Ein-/Ausgabe oder Arithmetik) zurückgreifen und diese wiederverwenden.

Bestandteile von Software

Software ist meist kein untrennbares Ganzes, sondern besteht aus einer Vielzahl von interagierenden Softwarekomponenten.

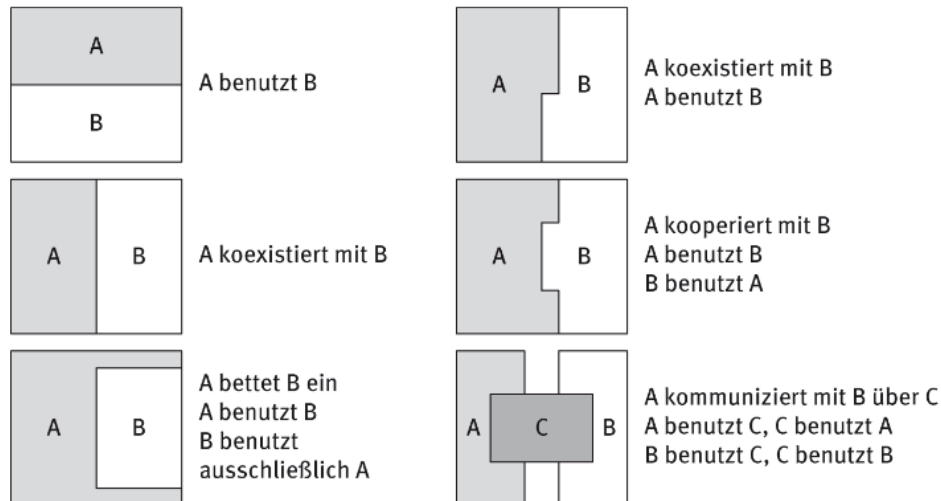


Welche Software könnte bei unserer vorstehenden Industrieanlage zum Einsatz gekommen sein?

Abhängigkeiten von Softwarekomponenten

Softwarekomponenten interagieren mit anderen Softwarekomponenten und bestimmen letztendlich die **Architektur** des Softwaresystems.

- Das Maß, in dem die verschiedenen Elemente innerhalb einer Softwarekomponente miteinander interagieren beziehungsweise miteinander verbunden sind, wird hierbei als **Kohäsion** (engl.: cohesion) der Komponente bezeichnet.
- Das Ausmaß, in dem eine Komponente mit anderen Komponenten interagiert, wird durch die **Kopplung** (engl.: coupling) gemessen.



Beim Architekturentwurf eines Systems wird angestrebt, die **Kohäsion** von Softwarekomponenten zu **maximieren**, während gleichzeitig die **Kopplung minimiert** werden soll. Auf diese Weise soll die Architektur des Systems möglichst leicht *verstehbar, nachvollziehbar und änderbar* gehalten werden.

Beispiel:

Wenn beispielsweise zwei Programmkomponenten sich gegenseitig Textbeschreibungen von durchzuführenden Operationen zuschicken, so ist dies eine lose Kopplung. Die Programmkomponenten kommunizieren über eine sehr einfache Schnittstelle, sie müssen sehr wenig über die Implementierung der anderen Komponente wissen. Allerdings ist das Analysieren des Textes aufwendig. Wenn die Komponenten sich gegenseitig aufrufen, so ist dies mit wesentlich weniger Rechenaufwand verbunden, die Komponenten müssen allerdings beispielsweise die Programmiersprache, die Datentypen und die Namen der Funktionen kennen. Wenn diese Programmkomponenten beispielsweise gemeinsame globale Datenstrukturen nutzen, wird die Kopplung noch höher.

Benutzungsdiagramme / Komponentendiagramme

Mit Hilfe von sogenannten Benutzungsdiagrammen können statische „benutzt“- (engl.: uses) und „besteht aus“- (engl.: consists of) Beziehungen zwischen **Komponenten** modelliert werden.

Benutzungsdiagramme bieten eine einfache aber bereits recht ausdrucksstarke Möglichkeit zur Darstellung dieser **Beziehungen**.

Aus dem Diagramm kann beispielsweise abgelesen werden, welche Komponenten voneinander unabhängig sind, und welche Komponenten von Änderungen in einzelnen Komponenten betroffen sind.

Die Notation sowie die wesentlichen Beziehungen, die in Benutzungsdiagrammen dargestellt werden können, sind o. g. Abb. visualisiert.

Komponenten versus monolithische (= keine Trennung) Softwareentwicklung

Salopp gesprochen folgt die komponentenorientierte Softwareentwicklung somit dem Baukastenprinzip, wobei verschiedene Bausteine prinzipiell beliebig zu neuen (größeren und mächtigeren) Bausteinen kombiniert werden können.

Die komponentenorientierte Softwareentwicklung hat ihren Ursprung unter anderem in der Erkenntnis, dass monolithische Systeme nur schwer wartbar, erweiterbar, testbar und wiederverwendbar sind. In der Regel müssen monolithische Systeme zu einem späteren Zeitpunkt entweder als Ganzes ausgetauscht werden, oder sie werden als Altsysteme in eine neue Umgebung eingebunden. Oft ist das letztgenannte Vorgehen notwendig, da für Altsysteme häufig keine oder eine nur sehr unvollständige Dokumentation existiert, und niemand mehr genau weiß, wie diese Systeme im Detail funktionieren.

Was sind die Nachteile von einem monolithischen IT-System?

Übungen

Nehmen Sie an, ein komplexes Softwaresystem, das 300.000 Zeilen Code umfasst, soll in einem Detail erweitert werden. Welche Kriterien können oder sollen bei der Auswahl der Programmiersprache für die Erweiterungsprogrammierung herangezogen werden?

Würden Sie eher Softwarekomponenten wiederverwenden oder neu entwickeln? Welche Gründe sprechen für und gegen die Wiederverwendung von Softwarekomponenten?

Was drückt ein Benutzungsdiagramm von Softwarekomponenten aus?

Fallbezogene Aufgabe

Eine große Universität mit mehr als 20.000 Studierenden setzt für die Unterstützung der Lehr- und Lernprozesse ein umfangreiches **E-Learning-System** ein, das die Studierenden und die Lehrenden beim Lernen/Lehren und Kommunizieren über das Internet unterstützt. Das System wird auf Basis von **Open-Source-Software** gemeinsam mit anderen Universitäten weiterentwickelt und bietet lehrveranstaltungsbezogene Information (Syllabus, Kalender, An- und Abmeldung von Lehrveranstaltungen und Prüfungen, Notenbuch) und Lernmaterialien (Folien, Kontrollfragen, Musterklausuren, Lecturecasts, Aufgaben, Lern-Apps), die Unterstützung der Kommunikation in Foren oder Wikis, sowie weitere Online-Services wie Lehrveranstaltungsverzeichnis, Prüfungseinsicht, Lehrveranstaltungsevaluierung, elektronische Einreichung, Speicherung und Plagiatsüberprüfung von Abschlussarbeiten. In der Zeit der Prüfungsvorbereitung lösen Studierende bis zu 600.000 Beispiele online, wobei bis zu 2.500 Studierende gleichzeitig aktiv sind. Die durchschnittliche Antwortzeit des Systems beträgt weniger als 0,1 Sekunden.

Die Systemkonfiguration besteht derzeit aus einem **Entwicklungs- und einem Produktionsserver** mit jeweils zwei Intel-Xeon-E5-2560 Prozessoren mit je 2,6 GHz. Jeder dieser Prozessoren hat acht Kerne. Zur Erhöhung der Ausfallsicherheit dienen noch zwei weitere Rechner mit identischer Ausstattung, die im Normalbetrieb für Zusatzdienste genutzt werden. Als Speichersystem wird ein universitätsweites **SAN-Speichersystem** von IBM genutzt, das über **Glasfaserkabel** mit den **Serverrechnern** verbunden ist. Die Rechner werden unter dem Betriebssystem Linux betrieben. Das E-Learning-System speichert die Daten im **relationalen Datenbanksystem PostgreSQL**. Der Umfang der Datenbank beträgt derzeit etwa 50 GB, wobei der Umfang pro Jahr etwa 10 GB zunimmt. Die etwa 1.200 Lecturecasts liegen im Dateisystem und belegen etwa 1,5 TB. Die von den Benutzern in das System hoch geladenen Dateien (beispielsweise Studienführer, Folien, Literatur, Dateiablage, Abgabe von Hausübungen) umfassen 1,9 TB mit einem Wachstum von etwa 300 GB/Jahr. Der skalierbare, Internet-basierte Zugriff wird über einen **Webserver namens NaviServer** realisiert, der über eine **interpretierte Sprache mit Anwendungslogik** erweiterbar ist und im laufenden Betrieb weiterentwickelt werden kann. Sowohl PostgreSQL als auch der NaviServer sind in einer kompilierten Sprache implementiert, um möglichst kurze Verarbeitungszeiten zu gewährleisten. Während PostgreSQL für die Abarbeitung von **SQL-Abfragen** mehrere Prozesse verwendet, sind beim NaviServer durchschnittlich etwa 50 **Threads** gleichzeitig mit der Abarbeitung von Benutzeranfragen beschäftigt. Das E-Learning-System benutzt ein **modulares Community-System** auf Open-Source-Basis, für das mehrere hundert **Erweiterungskomponenten** entwickelt wurden. Der Gesamtumfang beträgt etwa 2 Millionen Zeilen Programmcode, von denen etwa ein Drittel im *Learn@WU-System* im Einsatz ist. Die Basiskomponenten wurden weiterverwendet, zahlreiche Komponenten wurden weiterentwickelt, einzelne Komponenten wurden neu entwickelt.

Welche Rolle spielt Modularität und Wiederverwendung von Softwarekomponenten bei dem skizzierten System? Welche Problematiken könnten in einer Softwareentwicklung entstehen, wenn nicht modular entwickelt wird?